Long Short-term memory (LSTM)

Geek4Geek, IBM

LSTM architecture is a special type (an extension) of RNN designed to learn and remember information over long sequences of data.

So human, they learn, they remember, and when they read, they can understand the word that they remember

LSTM is used in:

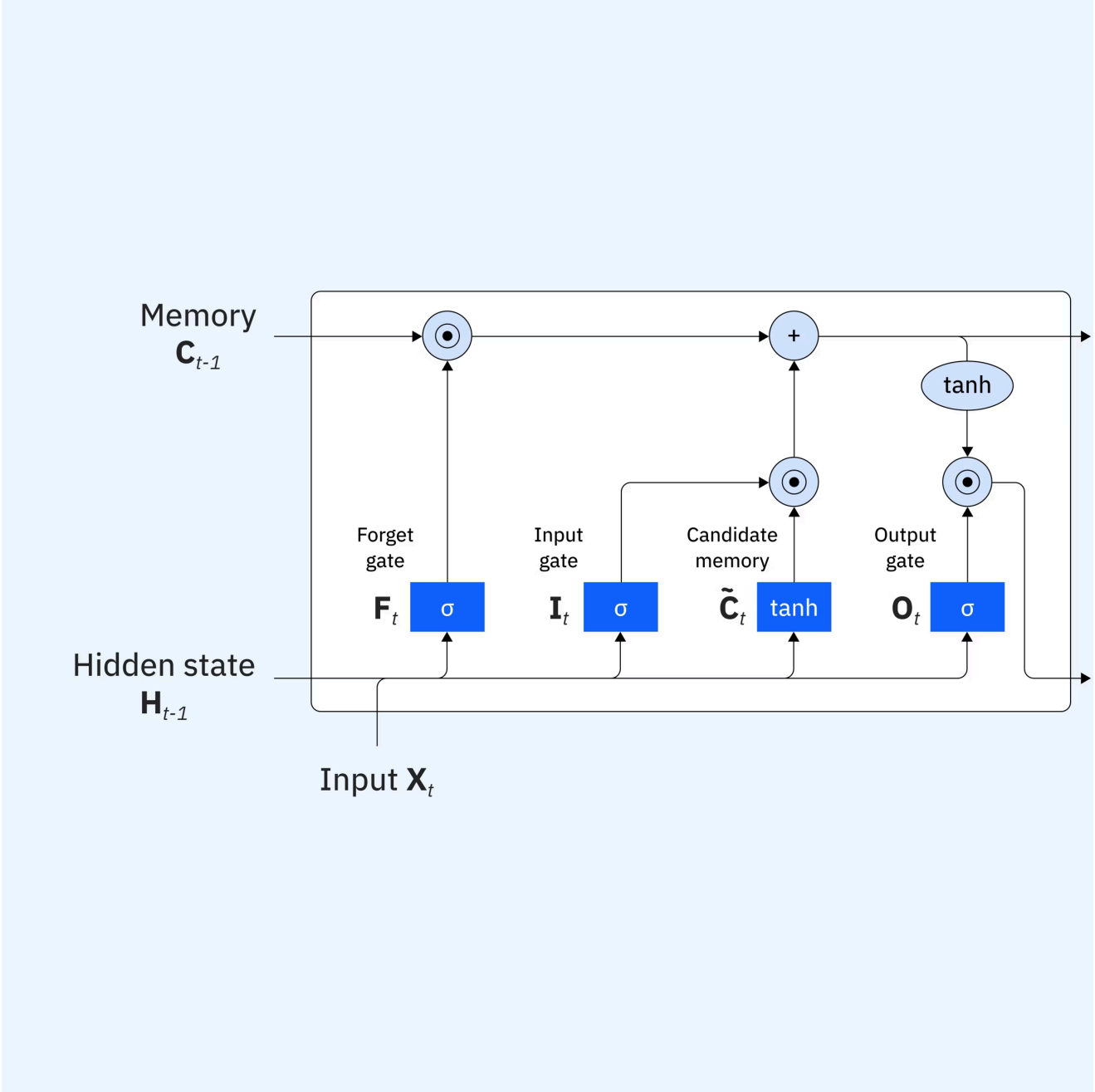
- **NLP**: language translation, speech recognition,
- **time series analysis**: stock market prediction, weather prediction, energy consumption
- **Anomaly detection**: detecting fraud or network intrusions. these network can identify the patterns in data that deviate drastically and flag them as potential anomalies.
- **Recommender systems**: it learn user behavior
- **Video analysis**: object detection, activity detection, action classification. when combined with CNNs, they help analyze video data and extract useful information

Problem with Long-Term Dependencies in RNN

So RNNs are designed to handle sequential data by maintaining a hidden state that captures information from previous time steps. but, they may face problem with long-term dependencies where information form distant time steps become crucial for making accurate predictions for current state. this problem is known as

- **Vanishing Gradient:** When training a model over time, the gradients which help the model learn can shrink as they pass through many steps. This makes it hard for the model to learn long-term patterns since earlier information becomes almost irrelevant.
- **Exploding Gradient:** Sometimes gradients can grow too large causing instability. This makes it difficult for the model to learn properly as the updates to the model become erratic and unpredictable.

How LSTMs work



LSTM architecture

it involves the **memory cell** which is controlled by three gates:

- **Input gate**: controls what information is added to the memory cell.
  - First the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using the inputs h_{t-1} and x_t. At the last the values of the vector and the regulated values are multiplied to obtain the useful information.
    - Equation for the input gate:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

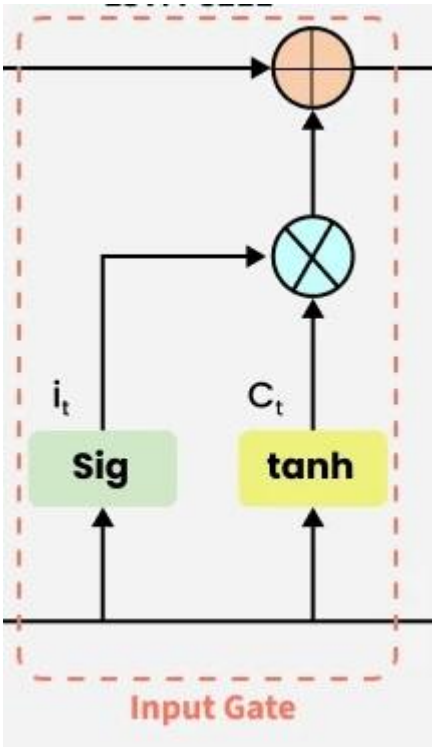
$$\hat{C}_t = tanh(W_c \cdot [h_{t-1}, x_t] + b_c$$

We multiply the previous state by f_t effectively filtering out the information we had decided to ignore earlier. Then we add $i_t \bigodot C_t$ which represents the new

candidate values scaled by how much we decided to update each state value

$$C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$$

where the $\odot$ denotes the element-wise multiplication

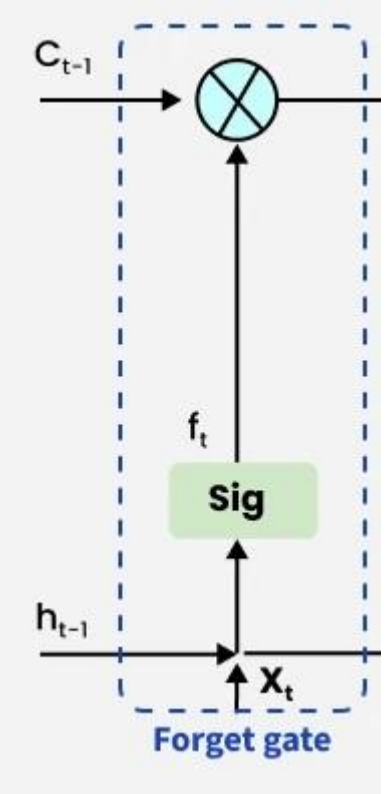


- **Forget gate**: Determine what information is removed (the information that is no longer useful in the cell state) from the memory cell.
  - input x_t at the particular time and input h_{t-1} precious cell output
    - after the input, we multiply with weight matrices followed by the addition of bias. Then the resultant is passed through sigmoid which give the out put between 0 and 1
      - 0 or near 0, the piece of information is forgotten
      - 1 or near 1, the piece of information is retrieved for the future use.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

where:

- W_f represents the weight matrix associated with the forget gate
- $[h_{t-1}, x_t]$ denotes the concatenation of the current input and the previous hidden state
- b_f is the bias with the forget gate
- σ is the sigmoid activation function



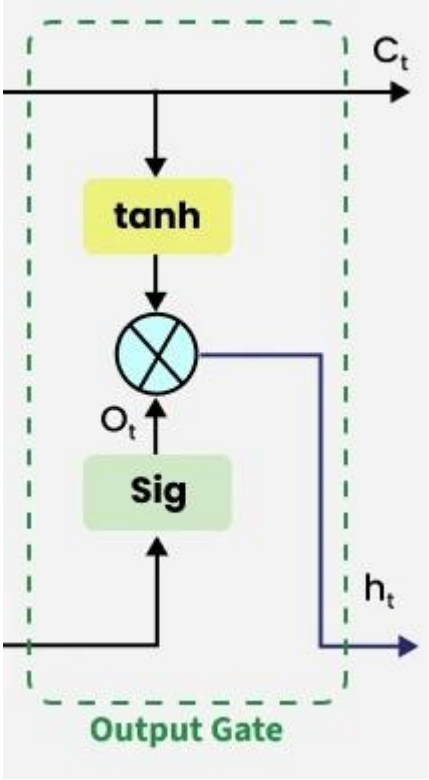
- **Output gate**: controls what information is output from the memory cell (it is responsible for deciding what part of the current cell state should be sent as the hidden state (output) for this time step). First it uses the sigmoid function to determine which info from the current cell state will be the output. This is done using the previous hidden state h_{t-1} and the current input x_t :

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Next, the current cell state C_t is passed through a tanh to scale its values between -1 and 1. Finally, this transformed cell state is multiplied element-wise with o_t to produce the hidden state h_t:

$$h_t = o_t \bigodot tanh(C_t)$$

where:
- o_t is the output gate activation
- C_t is the current cell state
- $\bigodot$ represents element-wise multiplication



This hidden state h_t is then passed to the next time step and can also be used for generating the output of the network

This will allow the LSTM network to selectively retain or discard information as it flows through the network which allow them to learn long-term memory.

This memory is updated using the current input, the previous hidden state and the current state of the memory cell